Data Structures

BS (CS) _Fall_2025

LAB 02 Tasks

Learning Objectives:

1. Class Templates with unit testing (Gtest)

Lab Overview

This lab focuses on understanding and implementing classes using templates in C++ along with unit testing using Google Tests.

Task 0: Simple Generic Holder

Create a template class Holder<T> that stores a single item of any type.

Data Members:

- **T item:** the stored value.

Functions:

- **Constructor:** initialize the item.
- **setItem(T value):** set the stored item.
- **getItem():** return the stored item.
- **displayType():** prints the data type of the stored item using typeid(item).name() from typeinfo header.

Test Cases:

- Create Holder<int>, Holder<double>, and Holder<string>.
- Set and get values for each.
- Print stored type and value.

Task 1:

Create a template class **MyMatrix** for performing various matrix operations with different data types e.g. int, double, float etc. This matrix class will handle same type of elements at a time and provide comprehensive matrix functionality. The **MyMatrix** class should consist of the following member variables:

T matrix A pointer to pointer (T**) representing a 2D dynamically allocated array to store matrix elements. **rows** An integer representing the number of rows in the matrix **cols** An integer representing the number of columns in the matrix

The MyMatrix class should consist of the following functions:

Constructor with rows and columns as parameters. This constructor should dynamically allocate memory for a 2D array and initialize all elements to zero.

Copy Constructor to create a deep copy of another MyMatrix object to avoid shallow copying issues.

Destructor to properly deallocate the dynamically allocated memory to prevent memory leaks.

- **inputMatrix()** which will take input from user to fill the matrix elements. This function should prompt the user to enter values row by row.

- **getElement(int row, int col)** which will return the element at specified position with proper bounds checking.
- **setElement(int row, int col, T value)** which will set the element at specified position with proper bounds checking.
- **printMatrix()** which will display the matrix in a properly formatted manner with appropriate spacing between elements.
- **isSquare()** which will return true if the matrix is a square matrix (rows == cols), otherwise false.
- **checkIdentity()** which will return true if the matrix is an identity matrix (square matrix with 1s on diagonal and 0s elsewhere), otherwise false. This function should first check if matrix is square.
- **addMatrix(const MyMatrix& other)** which will return a new MyMatrix object containing the sum of current matrix and the matrix passed as parameter. This function should check if both matrices have same dimensions before performing addition.
- **subtractMatrix(const MyMatrix& other)** which will return a new MyMatrix object containing the difference of current matrix and the matrix passed as parameter. This function should check if both matrices have same dimensions.
- **transpose()** which will return a new MyMatrix object containing the transpose of the current matrix. The transpose should have dimensions swapped (rows become cols and vice versa).

Test Cases:

Create instances for different data types (**int, float, double**) and perform the following operations for each type:

1. Create matrices of different sizes (2x2, 3x3, 4x3)
2. Display the matrices using printMatrix()
3. Test identity matrix checking with both identity and non-identity matrices
4. Test the isSquare function with square as well as non-square matrices
5. Demonstrate matrix addition, subtraction
6. Show transpose operation

Task 2:

You have to make a class for an online Banking system. The bank keeps a record of all the transactions made along with the current balance. The bank allows a specific number of transactions to be made.

You have to define a template class **OnlineBank** which can store different types of balance e.g.

int, double, float etc. Bank will have same type of elements at a time. The **OnlineBank** class should have the following member variables:

- **T currentBalance** the current balance in the bank.
- **T* withdrawals** A pointers to set the withdrawals made from the account. Before making a withdrawal, it should be checked if it's smaller than or equal to the current balance.
- **T* deposits** A pointers to set the deposits made from the account.
- **numOfTransactions** this is the number of withdrawals and deposits that can be made
- **counterWithdrawal** this is the number of current withdrawals made
- **counterDeposits** this is the number of current deposits made

The class should consist of following functions:

- **Constructor** with current balance and number of transactions as parameter
- **isWithdrawal** checks if the number of current withdrawals is less than the numOfTransactions
- **isDeposit** checks if the number of current deposits is less than the numOfTransactions
- **makeWithdrawal** which will return true if the value passed in the parameter is less than or equal to the current balance and its counter is less than the limit. It will deduct the given amount from current balance and update it.
- **makeDeposit** which will get a value to be added in the current balance if its counter is less than the limit
- **getCurrentBalance()** will return the current balance
- **print** this will print the current balance

Test Cases:

Create a comprehensive test class for the **OnlineBank** template class. Create separate instances for **int, float, double** data types and perform the following test cases:

- Create OnlineBank instances with different initial balances and transaction limits
- Test multiple valid deposits within transaction limits and verify balance updates
- Test multiple valid withdrawals where amount ≤ currentBalance and verify balance updates
- Attempt invalid withdrawals where amount > currentBalance and verify rejection

- Test transaction limits by exceeding both deposit and withdrawal limits
- Perform mixed deposits and withdrawals in random combinations
- Test boundary cases like zero balance, minimum amounts, and exact balance withdrawals
- Test data type specific operations (whole numbers for int, decimals for float/double)